Sparkle Movie 2

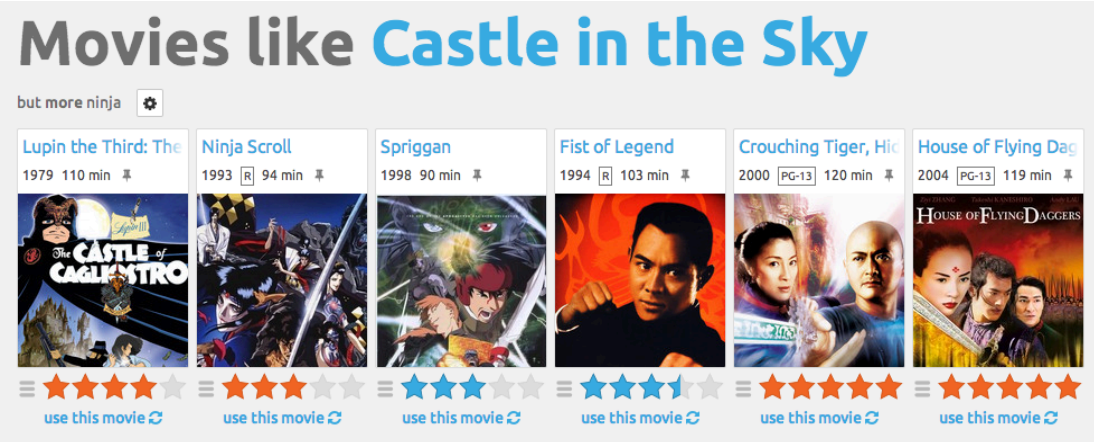
Apache Spark peut traiter des pétaoctets de données en quelques secondes en distribuant les tâches sur plusieurs machines ! ⚡

MovieLens

Vous travaillez pour une plateforme de streaming vidéo qui souhaite améliorer l'expérience utilisateur en proposant des recommandations personnalisées à grande échelle à ses abonnés.

Vous devez :

- Concevoir et entraîner plusieurs variantes de moteurs de recommandation à partir du dataset MovieLens.
- Industrialiser l'un de ces moteurs dans un environnement PySpark conteneurisé.
- Déployer un service d'inférence en conditions proches de la production, avec un monitoring simple de la charge et de la dérive du modèle.



Le dataset **MovieLens** contient les informations suivantes :

1. **ratings.csv** : Les notes attribuées par les utilisateurs aux films.
    - userId, movieId, rating, timestamp
2. **movies.csv** : Les métadonnées des films.
    - movieId, title, genres

Objectifs pédagogiques

Ce projet Sparkle Movie 2 s'adresse à deux types d'étudiants de niveau Master 1 :

(a) Ceux qui n'ont pas suivi le parcours Bachelor ou qui n'ont jamais travaillé sur les moteurs de recommandations, ni avec Spark : ils sont invités à **réaliser uniquement la première partie du projet**.

(b) Ceux qui ont suivi le parcours Bachelor et ont déjà réalisé le projet Sparkle Movie 1. Ces étudiants sont invités à **réaliser la partie 2 du projet**, sur la base de la partie 1 développée l'année précédente (réutilisation et évolution du dépôt GIT).

Selon l'environnement matériel disponible pour ce projet, une **machine de staging et de production** pourra être mise à la disposition des étudiants, afin de déployer le projet (partie 1 ou/et partie 2) en conditions industrielles.

À l'issue du projet, vous devrez être capables de :

- Concevoir, **comparer et évaluer plusieurs approches de recommandation** (collaborative, contenu, proximité).
- **Structurer un pipeline d'entraînement** reproductible pour un modèle de recommandation **sur Spark**.
- Construire une image **Docker intégrant PySpark** et votre moteur de recommandation.
- **Déployer un service d'inférence** sur un serveur et le paramétrer.
- Mettre en place un **monitoring simple de la charge** (latence, débit, erreurs) et de la **dérive des données / prédictions du modèle**.

Partie 1 : Moteurs de recommandation

1. Préparation de l'environnement

- Installez PySpark sur votre machine de développement ou configurez un environnement adapté (conda, venv, etc.).
- Configurez une session Spark et vérifiez son bon fonctionnement avec un mini job de test.
- Récupérez le dataset MovieLens depuis cette page, (taille au choix selon vos ressources).
- Créez un repository GitHub public nommé sparkle-movie qui contiendra tout votre travail (code, notebooks, Docker, documentation).

2. Chargement et exploration des données

- Importez les fichiers ratings.csv et movies.csv dans des DataFrames Spark.

- Affichez les 10 premières lignes de chaque DataFrame pour en comprendre la structure et vérifier le schéma.
- Nettoyez les données si nécessaire (valeurs manquantes, doublons, incohérences).
- Analysez les tendances générales, par exemple :
    - Films les mieux notés en moyenne (avec un seuil minimal de nombre de notes).
    - Genres les plus populaires en termes de volume de notes et de moyenne.
- Générez différentes visualisations à l'aide de librairies Python (matplotlib, seaborn, etc.) et de l'outil **Tableau Desktop**.

3. Modélisation avec Spark MLlib : ALS

- Utilisez l'algorithme **ALS (Alternating Least Squares)** de Spark MLlib pour entraîner un premier modèle de recommandation.
- Séparez vos données en train / validation / test.
- Ajustez les hyperparamètres clés :
    - rank, regParam, nombre d'itérations, éventuel filtrage des utilisateurs/films rares.
- Évaluez la performance du modèle avec une métrique adaptée (RMSE, MAE) sur le jeu de validation, puis sur le test.
- Identifiez une configuration de référence qui servira de base pour le déploiement.

4. Recommandation basée sur le contenu

- Créez des profils de films à partir des genres et éventuellement d'autres informations (titres, mots-clés simples).
- Représentez les films dans un espace vectoriel (par exemple TF-IDF sur les genres / tags).
- Implémentez un système recommandant des films similaires à ceux appréciés par un utilisateur, en utilisant une similarité (cosinus, distance, etc.).
- Comparez qualitativement ces recommandations avec celles du modèle ALS.

5. Recommandation Basée sur les Proximités Utilisateurs (KNN)

- Implémentez une approche basée sur les proximités entre utilisateurs (User-Based KNN) ou entre items (Item-Based KNN).
- Définissez une métrique de similarité (cosinus, corrélation, etc.).
- Générez des recommandations en fonction des évaluations des voisins proches.
- Discutez les avantages et limites par rapport à ALS et à l'approche contenu.

6. Evaluation des approches de recommandation

- Évaluez la précision et la couverture de vos recommandations (selon des métriques que vous aurez choisies et justifiées).
- Comparez les résultats des différentes approches :
    - Qualité des recommandations.
    - Complexité de mise en œuvre.
    - Temps de calcul et scalabilité.

- Proposez des recommandations concrètes **pour 3 à 5 utilisateurs fictifs** en expliquant les choix de films (quel modèle, pourquoi).
- Concluez sur la performance et la pertinence de vos méthodes.

Partie 2 : Industrialisation et Observation

Vous devez transformer votre moteur de recommandation en service déployable et monitoré.

7. Structuration du pipeline d'entraînement

1. Réorganisez votre code en plusieurs modules / scripts (pas seulement des notebooks) :
    - `data_loader.py` : chargement et préparation des données.
    - `train.py` : entraînement du modèle (ALS ou autre modèle choisi).
    - `evaluate.py` : évaluation et comparaison des modèles.
2. Paramétrez votre pipeline via un fichier de configuration (`config.yaml` ou équivalent) ou via des variables d'environnement : chemins des données, hyperparamètres, taille des échantillons, etc.
3. À chaque exécution d'entraînement, enregistrez :
    - Les hyperparamètres utilisés.
    - La date et l'heure.
    - Les métriques de performance (RMSE, MAE, etc.).
    - Le chemin de sauvegarde du modèle.

8. Service d'inférence (API)

1. Implémentez un service d'API (Flask ou FastAPI) qui :
    - Charge au démarrage le modèle entraîné (ou les artefacts nécessaires).
    - Expose au minimum une route `/recommendations?user_id=...` renvoyant une liste de films recommandés.
    - Gère les cas d'erreur (utilisateur inconnu, absence de modèle, etc.).
2. Ajoutez une route `/health` qui permet de vérifier l'état du service (modèle chargé, Spark opérationnel, etc.).
3. Documentez les endpoints (paramètres attendus, format des réponses) dans votre `README` ou un fichier dédié (OpenAPI/Swagger optionnel).

9. Conteneurisation avec Docker

1. Créez un `Dockerfile` pour construire une image intégrant :
    - Python + PySpark + Java (ou une image Spark de base).
    - Vos dépendances Python (`requirements.txt`).
    - Votre code (modules, scripts, API).
2. Configurez les variables d'environnement nécessaires à Spark (mémoire, nombre de threads, logs).
3. Définissez une commande de démarrage (`CMD` / `ENTRYPOINT`) lançant le service d'API.
4. Ajoutez un test rapide (smoke test) dans votre process de build ou de démarrage pour valider que Spark peut exécuter un job simple dans le conteneur.

10. Déploiement sur un serveur distant

1. Déployez votre image sur un serveur de “production pédagogique” (VM, machine distante, etc.). Si une telle machine n’est pas disponible, vous utiliserez votre PC.
2. Lancez votre conteneur avec une configuration explicite :
    - Volumes pour les données, les modèles et les logs.
    - Ressources limitées (CPU, RAM) à définir et justifier.
3. Documentez précisément la procédure de déploiement dans un fichier `DEPLOYMENT.md` :
    - Comment construire l’image.
    - Comment la pousser / transférer.
    - Comment lancer le conteneur avec les bons paramètres.

11. Monitoring de charge et de dérive

1. Mettez en place un **module de monitoring léger** (dans le service ou dans un script séparé) qui enregistre pour chaque requête :
    - Timestamp, `userId`, latence de la requête, nombre d’items recommandés, éventuelle erreur.
2. Calculez des métriques agrégées sur une fenêtre temporelle (par exemple 5 minutes ou 1 heure) :
    - Latence moyenne et p95.
    - Nombre de requêtes, taux d’erreurs.
3. Pour la dérive des données / prédictions :
    - Définissez une distribution de référence (baseline) pour une ou deux grandeurs simples, par exemple :
        - Répartition des genres recommandés.

    - Moyenne des scores de recommandation.
  - Comparez régulièrement la distribution courante à la baseline (écarts en pourcentage, indice de stabilité de population simple, test de KS si vous le souhaitez).
  - Définissez des seuils d'alerte symboliques (par exemple "alerte si écart > 20% sur la proportion d'un genre").
4. Présentez un court rapport ou quelques graphiques montrant l'évolution de ces métriques sur une période de test (simulation de trafic possible via un script).

Recommandations techniques pour le **Monitoring de charge**

Minimum obligatoire : logs + métriques basiques; logging structuré Python suffisant pour compter les requêtes, mesurer les temps de réponse et écrire dans un CSV / JSON, puis analyser offline (Spark/pandas) dans un petit notebook de monitoring.

Optionnel : exposition Prometheus et (encore plus optionnel) dashboard Grafana. prometheus_client : une seule librairie à ajouter pour exposer un endpoint /metrics avec compteur de requêtes, histogramme de latence, etc.

Recommandations techniques pour le **Monitoring de dérive modèle / données**

Librairie recommandée pour la dérive : Evidently (un rapport de drift minimal sur 1–2 features suffit). Génération de rapports de drift à partir de deux DataFrames (référence vs courant). Vous pouvez brancher ça sur les recommandations retournées (genres, scores moyens) et produire un HTML ou un notebook de rapport.

Alternative ultra-light : un petit module maison qui recalcule distributions et KS, mais Evidently vous fait gagner du temps pour le rendu visuel.

Compétences visées

Partie 1 : Consolidation et Machine-Learning-Ops

- ➔ Analyse de données à grande échelle avec Spark.
- ➔ Conception et comparaison de moteurs de recommandation.
- ➔ Mise en œuvre d'un pipeline MLOps basique : entraînement, évaluation, versionning du modèle.

Partie 2 : Industrialisation et Observation

- ➔ Conteneurisation (Docker) et déploiement d'un service de data / IA.
- ➔ Mise en place de métriques de monitoring (charge, dérive) et interprétation des résultats.

Rendu

L'évaluation de ce projet se fera sur plusieurs aspects :

1. Une **présentation de votre travail sous forme de diapositives** expliquant :
    a. Le contexte et les données.
    b. Les différentes approches de recommandation testées et leurs résultats.
    c. L'architecture de votre solution déployée (schéma, choix techniques, limites).

d. Votre stratégie de monitoring et les enseignements tirés (si votre groupe à réalisé la partie 2).

2. Un repository github public nommé **sparkle-movie**, contenant les éléments suivants :

   a. Un **notebook Python** **propre et commenté** (introduction, titres des sections, interprétation des visuels, conclusion, etc) contenant le procédé de développement de votre outil, du nettoyage à la modélisation des données, en passant par l'**analyse exploratoire.** **Pensez à répondre à la problématique.** Vous pouvez avoir au maximum deux notebooks, un pour l'exploration et l'autre pour la modélisation de données.

   b. Le **code structuré du pipeline** (scripts de data, entraînement, évaluation, service d'API).

   c. Un `README.md` complet : contexte, données, modèles, architecture, instructions de lancement.

3. Pour ceux qui auront développé la partie 2 du projet:

   a. Le `Dockerfile`, éventuellement un `docker-compose.yml`.

   b. Un `DEPLOYMENT.md` : procédure détaillée pour construire et déployer l'image sur un serveur.

   c. Éventuellement un dossier `monitoring` contenant scripts / notebooks d'analyse des métriques.

Base de connaissances

- PySpark Overview
- Collaborative Filtering – Spark ML ALS (doc officielle)
- Qu'est-ce que Tableau Desktop ?
- FastAPI in Containers – Docker
- Prometheus Python client – Official Documentation
- How to start with ML model monitoring. A step-by-step guide.